Member-only story

10 Git Commands Every Developer Should Know but Doesn't Use Enough

Stop suffering through git workflows — these commands will change your life

8 min read · Nov 9, 2025

Bhavyansh Follow

We all know `git add`, `git commit`, and `git push`. We probably even know `git merge` and `git rebase` (though we might not admit which one we actually use).

Don’t mind the awkward hand growth, ai images don’t have git features

I’ve been using Git for almost a decade, and I only discovered some of these commands last year. Each one made me think, “Where have you been all my life?”

So here are ten Git commands that deserve way more love than they get.

1. `git add -p` — The "I Changed Too Many Things" Savior

You know the drill. You sit down to fix one bug. Three hours later, you've refactored half the codebase, fixed two other bugs, and added a feature that wasn't even on the roadmap.

Now you need to commit. Do you dump everything into one massive commit? Create three commits with careful manual staging? Rage-quit and grab coffee?

Enter `git add -p` (patch mode):

```
git add -p
```

This command walks you through every change, chunk by chunk, and asks: "Want to stage this?"

```
diff --git a/app/models/user.rb b/app/models/user.rb
index 1234567..abcdefg 100644
--- a/app/models/user.rb
+++ b/app/models/user.rb
@@ -10,7 +10,7 @@ class User < ApplicationRecord
   validates :email, presence: true

-  def full_name
-    "#{first_name} #{last_name}"
+  def display_name
+    "#{first_name} #{last_name}".strip
   end

Stage this hunk [y,n,q,a,d,s,e,?]?
```

Your options:

- `y` — yes, stage this
- `n` — nope, skip it
- `s` — split this into smaller chunks
- `e` — let me manually edit what to stage
- `q` — I'm done, quit

I use this daily. It transforms messy work sessions into clean, logical commits without requiring perfect discipline upfront.

Pro tip: If you've already staged stuff you want to unstage, use `git reset -p` to selectively unstage chunks.

2. `git commit --fixup` — The Time Traveler's Tool

Ever push a commit, then immediately spot a typo? Or realize you forgot one file that belongs with that commit?

Most people create a new commit like "fix typo" or "forgot this file." These commits are noise.

Instead:

```
# Make your fix
git add fixed_file.rb

# Reference the commit you want to fix (use any identifier)
```

```
git commit --fixup HEAD
# Or fixup a specific commit
git commit --fixup abc1234
```

This creates a commit with the message `fixup! Original commit message`. Later, when you're ready:

```
git rebase -i --autosquash main
```

Git automatically reorders and marks the fixup commits to be squashed into their targets. Clean history, zero effort.

I pair this with `git log --oneline` to find commit hashes:

```
git log --oneline -10
```

This workflow changed my life. I used to stress about getting every commit perfect before pushing. Now I fix things as I notice them and clean up before merging.

3. `git stash -u` — The "Oh Crap, Wrong Branch" Escape Hatch

Classic scenario: You've been coding for 30 minutes. Then you realize you're on `main`. Or the wrong feature branch. Or you need to quickly check out another branch to review something.

Regular `git stash` works, but only for tracked files. If you created new files

(which you probably did), they don't get stashed.

```
# Stashes everything, including untracked files
git stash -u

# Or if you want ignored files too
git stash -a
```

Now you can:

```
# Switch to the right branch
git checkout feature/actual-branch

# Get your work back
git stash pop
```

Even better, name your stashes:

```
git stash push -u -m "WIP on user authentication"

# Later, see what you stashed
git stash list
# Apply a specific stash
git stash apply stash@{1}
```

I keep multiple stashes around for different contexts. It's like having multiple desks you can switch between instantly.

4. `git log -S` — The Code Archaeology Command

You need to find when someone deleted a function. Or added a specific string. Or changed a particular configuration value.

Scrolling through commits? Tedious. Searching GitHub? Limited.

Use pickaxe:

```
# Find commits that added or removed "authenticate_user"
git log -S "authenticate_user"

# Make it pretty
git log -S "authenticate_user" --oneline --all
```

This shows every commit that changed the number of occurrences of that string. It's incredible for:

- Finding when a function was deleted
- Tracking down who added a magic number
- Discovering why a configuration changed

Want to search for regex patterns instead? Use `-G`:

```
# Find commits that touched lines matching this pattern
git log -G "def.*authenticate" --patch
```

Add `--patch` to see the actual changes:

```
git log -S "DATABASE_URL" --patch
```

This is how you become a Git detective.

5. `git bisect` — The Bug Hunter's Secret Weapon

You know a bug exists in production. You know it didn't exist a month ago. You have 200 commits in between.

Do you manually check out commits until you find the culprit? No. You use `git bisect`.

```
# Start bisecting
git bisect start

# Mark current commit as bad (has the bug)
git bisect bad
# Mark a commit you know is good (before the bug)
git bisect good v1.2.0
```

Git checks out a commit halfway between good and bad. You test it:

```
# If the bug exists
git bisect bad

# If it doesn't
git bisect good
```

Git keeps bisecting until it finds the exact commit that introduced the bug. Binary search through your history.

```
Bisecting: 50 revisions left to test
Bisecting: 25 revisions left to test
Bisecting: 12 revisions left to test
abc1234 is the first bad commit
```

You can even automate it:

```
git bisect start HEAD v1.2.0
git bisect run npm test
```

Git runs your test command on each commit and automatically finds the breaking change. I've used this to track down bugs in massive codebases in under five minutes.

6. `git worktree` — Multiple Branches, Simultaneously

Ever needed to work on two branches at once? Maybe compare implementations, or quickly test something without losing your current work?

Most people stash, switch, unstash. Or clone the repo twice.

There's a better way:

```
# Create a new worktree for a different branch
git worktree add ../myproject-hotfix hotfix/critical-bug

# Now you have two directories, same repo, different branches
cd ../myproject-hotfix
# Work on the hotfix
# Back to main work
cd ../myproject
```

Each worktree is a separate checkout of your repo. They share the same `.git` directory but have independent working trees.

List your worktrees:

```
git worktree list
```

Delete when done:

```
git worktree remove ../myproject-hotfix
```

This is perfect for:

- Emergency hotfixes while keeping feature work intact
- Comparing implementations side by side
- Running long-running tasks (builds, tests) in one worktree while coding

in another

I keep a worktree for `main` that I never touch—just for reference and quick deploys.

7. `git reflog` — The "Undo Literally Anything" Command

You ran `git reset --hard` and deleted work. Or rebased incorrectly. Or force-pushed the wrong thing.

Don't panic. Git keeps a log of everything your HEAD has pointed to:

```
git reflog
```

You'll see:

```
abc1234 HEAD@{0}: commit: Add user authentication
def5678 HEAD@{1}: rebase finished
ghi9012 HEAD@{2}: commit: Fix validation bug
jkl3456 HEAD@{3}: checkout: moving from main to feature
```

Want to undo that disastrous rebase? Find the commit before it started:

```
git reset --hard HEAD@{2}
```

Deleted a branch by accident? Find it in reflog:

```
git reflog
# Find the branch's last commit
git checkout -b recovered-branch abc1234
```

`reflog` is your safety net. Git almost never truly deletes data—it just makes it harder to find. Reflog finds it.

Warning: Reflog entries expire after 90 days by default. Don't rely on this forever.

8. `git rebase -i --autosquash` — Clean History on Autopilot

We talked about `--fixup` earlier. The autosquash rebase is where the magic happens.

Let's say you have:

```
git log --oneline
abc1234 Add login endpoint
def5678 fixup! Add login endpoint
ghi9012 Add user model
jkl3456 fixup! Add user model
mno7890 Initial commit
```

Normally, interactive rebase means manually reordering and squashing. But:

```
git rebase -i --autosquash mno7890
```

Git automatically reorders fixup commits to be right after their targets and marks them for squashing. You just save and close the editor.

Result:

```
git log --oneline
abc1234 Add login endpoint
ghi9012 Add user model
mno7890 Initial commit
```

Clean history, zero manual work.

Set it as default:

```
git config --global rebase.autoSquash true
```

Now `git rebase -i` automatically handles your fixups.

9. `git blame -w -C` — Smart Blame

Everyone knows `git blame`. But basic blame is noisy. It shows you who moved code, who fixed indentation, who added whitespace.

You want to know who wrote the logic, not who ran Prettier.

```
# Ignore whitespace changes
git blame -w file.rb
```

```
# Detect code moved or copied within the file
git blame -w -C file.rb
# Detect code moved from other files
git blame -w -C -C file.rb
# Detect code from any commit (slower)
git blame -w -C -C -C file.rb
```

This cuts through the noise. Someone copied a function from another file? Blame shows the original author, not the copier.

You can also see blame for deleted lines:

```
# Show what line 42 used to be
git blame -L 42,42 abc1234^ -- file.rb
```

The `^` means "parent commit." This shows who wrote the line before it was deleted in commit `abc1234`.

10. `git cherry-pick -x` — Selective Commit Porting

You need a specific commit from another branch. Just that one commit.

```
# Apply commit abc1234 to current branch
git cherry-pick abc1234
```

But add `-x` to record where it came from:

```
git cherry-pick -x abc1234
```

This adds a line to the commit message:

```
(cherry picked from commit abc1234567890abcdef1234567890)
```

Why this matters: Six months from now, when you're debugging, you'll see this commit exists in two branches and understand why.

Cherry-pick multiple commits:

```
git cherry-pick abc1234..def5678
```

Had a conflict and want to abort?

```
git cherry-pick --abort
```

Want to cherry-pick but not commit yet?

```
git cherry-pick -n abc1234
# Make additional changes
```

```
git commit
```

This is perfect for:

- Backporting bug fixes to release branches
- Pulling hotfixes from main into feature branches
- Applying experimental commits from one branch to another

Bonus: Combine Them

The real power comes from chaining these together:

```
# Find when a function was deleted
git log -S "old_function" --oneline

# Bisect to find the breaking commit
git bisect start HEAD abc1234
git bisect run npm test

# Create a worktree to fix it without losing current work
git worktree add ../fix-branch main
# Make fix, use fixup commits
git commit --fixup def5678
# Clean up with autosquash
git rebase -i --autosquash main

# Cherry-pick the fix to release branch
git checkout release-1.0
git cherry-pick -x fix-commit
```

Start Small

Don't try to memorize all of these. Pick one that solves a problem you face

regularly:

- Messy commits? → `git add -p`
- Lost work? → `git reflog`
- Need to find a bug? → `git bisect`
- Wrong branch? → `git stash -u`

Use it for a week. Make it muscle memory. Then add another.

Git has been around since 2005. These commands have been battle-tested by millions of developers. They exist because they solve real problems.

Stop fighting Git. Start using it properly. Your commit history — and your

team — will thank you.

Git | Version Control | Programming | Productivity | Developer Tools

Follow

Written by Bhavyansh

1.5K followers · 3 following

Hey I write about Tech. Join me as I share my tech learnings and insights. 🚀 Building flux8labs.com

Responses (2)

To respond to this story,
get the free Medium app.

Continue in app

Vinícius A. Goulart
3 days ago

There is a version of this post for non-members?

1 1 reply

Lee Doolan
13 hours ago

The command

git stash apply stash@{1}

can be cast as

git stash apply 1

More from the list: "GIT Gold"

Curated by Pancake Agree

In Stackad... by Subodh S...

15 Git Terms That Confuse Developers (an...

Oct 21, 2025

In Level Up ... by Pushkar ...

6 Commands That Quietly Changed How I...

Feb 14

View list

More from Bhavyansh

Bhavyansh

The Pattern I Notice in Every High-Quality Codebase

It's hiding in plain sight, and you've probably already seen it today.

Feb 7 2.3K 66

Bhavyansh

The 5-Minute Habit That Separates Senior Engineers

The unsexy practice that separates senior engineers from everyone else

Feb 13 403 8

Bhavyansh

I Finally Learned How Senior Engineers Read Code (It’s Not What You Think)

The mental models and reading strategies that separate experienced engineers from the rest

Dec 6, 2025 681 26

Bhavyansh

Why I Ignore Architecture Diagrams in System Design Reviews

Everyone obsesses over architecture diagrams. I look at something completely different.

Feb 23 55 5

See all from Bhavyansh

Recommended from Medium

Reza Rezvani

10 Claude Code Commands That Cut My Dev Time 60%: A Practical Guide

Custom slash commands, subagents, and automation workflows that transformed my team's productivity—with copy-paste templates you can use

Nov 20, 2025 1.91K 35

Nov 20, 2025 1.91K 30

In ITNEXT by Jacob Ferus

Cursor Is Dying

Cursor is a great product. It was one of the first great applications of AI in coding, moving past copy-pasting code from chats. But the AI...

Feb 11 575 30

Phil | Rentier Digital

I Stopped Vibe Coding and Started “Prompt Contracts” — Claude Code Went From Gambling to Shipping

Last Tuesday at 2 AM, I deleted 2,400 lines of code that Claude Code had just generated for me.

Feb 10 3.3K 90

In AI Software Engineer by Joe Njenga

Claude Can Now Create Complex Diagrams — I Tested 21 Prompts (Goodbye Canva!)

We have come along with Canva, but now Claude's new update does the job faster.

Mar 12 810 8

The Latency Gambler

Anthropic Says Engineers Won’t Exist in a Year. It’s Also Paying Them $570K Today.

The most honest job posting in tech history might also be the most revealing thing about where the industry actually stands.

Mar 11 325 12

Michael Swengel

This Linux Distro May Be the PERFECT Windows 11 Replacement

Form and function in one elegant package? Sign me up.

Mar 3 975 15

See more recommendations